

Understanding Your React Currency Converter

A complete, line-by-line walkthrough of every file — for someone who followed a tutorial and now wants to actually understand the code.

This app lets a user type an amount, pick a **from** currency and a **to** currency, and see the converted value. It also has a **swap** button to flip the two currencies instantly. Under the hood it is built from four files, each with one clear job. These notes go through **why** each file exists, **what** every line does, and **how** the pieces talk to each other.

File	What it is	Its one job
<code>main.jsx</code>	Entry point	Boots up React and mounts <code><App/></code> onto the HTML page
<code>App.jsx</code>	Parent / "smart" component	Owns all the state and business logic (amount, currencies, convert, swap)
<code>InputBox.jsx</code>	Child / "dumb" component	A reusable amount + currency-picker box, used twice
<code>useCurrencyInfo.js</code>	Custom hook	Fetches live exchange rates from an API and hands back a plain object

1. The Big Picture First

Before reading any code, it helps to see how the files are connected. React apps are built as a **tree of components** — think of components as custom, reusable HTML tags that you design yourself. Here, the tree looks like this:

```
main.jsx
  \--- <App />                                (the whole screen)
        |--- <InputBox label="From" .../>      (top box: amount you're converting FROM)
        |--- swap button
        |--- <InputBox label="To" .../>        (bottom box: shows the result)
        \--- uses the hook: useCurrencyInfo(from)
```

App.jsx is the "parent" — it holds all the data (state) and all the logic. **InputBox.jsx** is a "child" component — it does not decide anything by itself, it just displays whatever data App gives it, and reports back to App when the user types or picks something. **useCurrencyInfo.js** is not a component at all — it's a small reusable function (a "hook") that App calls to go fetch exchange-rate data from the internet.

Why split the code into separate files at all?

Because each file should have ONE job (this is called **separation of concerns**). InputBox is written once and reused twice (From box and To box) instead of copy-pasting the same JSX twice. The data-fetching logic lives in its own hook so it can be reused by any component, tested on its own, and kept out of App's way. This is standard React practice, not something specific to this tutorial.

2. main.jsx — The Entry Point

Every React app needs exactly one place where it "switches on." This file is that place. It is the very first JavaScript that runs when your app loads in the browser.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

Line by line

- **import { StrictMode } from 'react'** — StrictMode is a special wrapper component that doesn't render anything visible. It just tells React: "run in development in a stricter mode and warn me about bad practices." In development, it can even call some functions twice on purpose, just to help you catch bugs (this is why, if you ever notice a console.log firing twice, it's not a bug — it's StrictMode checking you).

- **import { createRoot } from 'react-dom/client'** — **react** (the core library) knows how to build components and manage state, but it doesn't know how to draw anything on screen. **react-dom** is the part that knows how to translate React components into real HTML elements in the browser. **createRoot** is the modern function used to connect React to a real DOM node.
- **import './index.css'** — pulls in your global stylesheet (this is typically where Tailwind's base styles get imported in a Tailwind + Vite project).
- **import App from './App.jsx'** — brings in your main component so it can be rendered.
- **document.getElementById('root')** — every Vite/React project has an **index.html** file with a line like `<div id="root"></div>`. This single line grabs that empty `<div>` from the real HTML page.
- **createRoot(...).render(<App />)** — this is the command that says: "take my React component tree, starting at `<App/>`, and inject the actual HTML it produces into that empty div." From this point on, React takes over that div completely and manages everything inside it.

Why does this matter for you?

You will basically never touch `main.jsx` again once it's set up. Its only job is the handoff from "plain HTML page" to "React-controlled page." All real logic and UI live in `App.jsx` and below.

3. useCurrencyInfo.js — Your Custom Hook

This file's only job is: **"given a currency code like 'usd', go fetch today's exchange rates for it, and hand them back."** It is written as a **custom hook**, which is one of the most important patterns in React.

What is a "hook" at all?

In React, a **hook** is simply a function whose name starts with `use`, and which is allowed to use React's special features — like remembering values between renders (`useState`) or running side-effects like network calls (`useEffect`). A **custom hook** is just your own function, also named `useSomething`, that internally calls one or more of React's built-in hooks so you can package up and reuse some logic.

```
import { useEffect, useState } from "react";

function useCurrencyInfo(currency) {
  const [data, setData] = useState({});
  useEffect(() => {
    fetch(`https://cdn.jsdelivr.net/npm/@fawazahmed0/currency-api@latest/v1/currencies/${currency}.j
      .then((res) => res.json())
      .then((res) => setData(res[currency]))
    }, [currency])
  return data
}
export default useCurrencyInfo
```

Line by line

- **function useCurrencyInfo(currency)** — it takes one input: the currency code you want rates FOR (e.g. "usd"). It will return an object of conversion rates, like `{ inr: 83.2, eur: 0.91, gbp: 0.79, ... }`.

- **`const [data, setData] = useState({})`** — this is React's memory box. `data` starts as an empty object `{}`. `setData` is the only way you're allowed to change it. Every time `setData` is called with a new value, React automatically re-renders any component using this hook, so the screen updates.
- **`useEffect(() => {...}, [currency])`** — `useEffect` lets you run code that reaches "outside" React (like a network request) at the right moment, instead of during rendering. The array at the end, `[currency]`, is the **dependency array**: it tells React "only re-run this block again if the value of `currency` has changed since last time." So the very first time the component using this hook appears, it fetches once. Later, if the user changes the "From" currency dropdown, `currency` changes, and this effect automatically fires again to fetch fresh rates for the new currency.
- **`fetch(`.../currencies/${currency}.json`)`** — this calls a free public currency-exchange-rate API. The backtick string is a **template literal**: whatever is inside `${...}` gets substituted in, so if `currency` is "usd", the actual URL becomes `.../currencies/usd.json`.
- **`.then((res) => res.json())`** — `fetch()` first gives you a raw HTTP response. `res.json()` reads the body of that response and parses it from JSON text into a real JavaScript object. This step is itself asynchronous (it also returns a promise), which is why it needs its own `.then()`.
- **`.then((res) => setData(res[currency]))`** — the API actually responds with something shaped like `{ usd: { inr: 83.2, eur: 0.91, ... } }` — the outer key repeats the currency you asked about. So `res[currency]` reaches one level in and pulls out just the inner rates object, which is then saved into state.
- **return data** — whatever component calls this hook receives the current rates object. The first time it's called (before the fetch finishes) it will be `{}`; a moment later, once the network call completes, the component re-renders automatically with real data.

Why not just write this `fetch()` code directly inside `App.jsx`?

You could — it would still work. But wrapping it in a hook gives you three benefits: (1) **Reusability** — any other component in a bigger app could call `useCurrencyInfo('eur')` and get the same behaviour for free. (2) **Readability** — `App.jsx` stays focused on layout and user actions instead of being cluttered with `fetch/promise` code. (3) **Testability** — you can test the data-fetching logic in isolation from the UI.

4. `InputBox.jsx` — The Reusable Building Block

This component draws one white rounded box containing an amount field on the left and a currency dropdown on the right. It is used **twice** in `App.jsx` — once for the "From" box, once for the "To" box — with different data plugged in each time.

```
function InputBox({
  label,
  amount,
  onAmountChange,
  onCurrencyChange,
  currencyOptions = [],
  selectedCurrency = "usd",
  amountDisable = false,
  currencyDisable = false,
  className = "",
}) {
  const amountInputId = useId()
  ...
}
```

Why is this a separate file instead of just writing it twice inside App.jsx?

This is the **DRY principle** — "Don't Repeat Yourself." Both boxes look and behave identically (an amount input + a currency dropdown); only the actual values differ. Instead of duplicating the same chunk of JSX twice and having to fix bugs in two places, it's written once as a component and simply called twice with different **props**.

What are "props"?

Props (short for *properties*) are just the inputs you pass into a component — exactly like arguments you pass into a function. When App writes `<InputBox label="From" amount={amount} .../>`, it is handing InputBox a bunch of values it needs to display and use. InputBox itself never decides what the amount IS — it just displays whatever amount it was given, and tells its parent when the user changes something.

Every prop, explained

- **label** — the small grey text ("From" / "To") shown above the amount field.
- **amount** — the numeric value to display and type into.
- **onAmountChange** — a function (given by the parent) to call whenever the user types a new amount. InputBox itself doesn't store this value — it just reports it upward.
- **onCurrencyChange** — same idea, but fired when the user picks a different currency from the dropdown.
- **currencyOptions = []** — the list of currency codes to show inside the dropdown, e.g. ["usd", "inr", "eur", ...]. The `= []` is a **default value** — if the parent forgets to pass this prop, it defaults to an empty array instead of crashing.
- **selectedCurrency = "usd"** — which option in the dropdown is currently selected; defaults to "usd" if not given.
- **amountDisable = false** — when true, greys out and locks the amount input so the user can't type into it. Used on the "To" box, because that field should only ever show a calculated result, never be typed into directly.
- **currencyDisable = false** — same idea but for locking the currency dropdown (not actually used in your App.jsx right now, but available if needed later).
- **className = ""** — lets the parent optionally tack on extra Tailwind classes for one specific usage, without editing this file.

useId() — why does the input need an id at all?

`const amountInputId = useId()` calls a React hook that generates a guaranteed-unique string, safe to use as an HTML id. It's needed because of accessibility: the `<label>` tag has `htmlFor={amountInputId}`, and the `<input>` has `id={amountInputId}`. Matching these two lets screen readers and "click the label to focus the input" behaviour work correctly. Since `InputBox` is rendered **twice** on the same page (From and To), a hard-coded id like `id="amount"` would create two elements with the exact same id — which is invalid HTML and breaks that label-input link. `useId()` guarantees each copy of the component gets its own unique id automatically.

The controlled input pattern

```
<input
  id={amountInputId}
  type="number"
  disabled={amountDisable}
  value={amount}
  onChange={(e) => onAmountChange && onAmountChange(Number(e.target.value))}
/>
```

- **`value={amount}`** — the input's displayed value is always exactly whatever `amount` prop it was given. This means React, not the browser, is the "source of truth" for what's in the box. This pattern is called a **controlled component**.
- **`onChange={(e) => ...}`** — every single keystroke fires this. `e` is the browser's event object; `e.target.value` is the raw text currently in the box (always a string, even for number inputs). `Number(...)` converts that string into an actual number before sending it up.
- **`onAmountChange && onAmountChange(...)`** — this is a safety check. If the parent didn't bother to pass an `onAmountChange` function (like in the "To" box, which doesn't need one since it's disabled), this line quietly does nothing instead of crashing with "onAmountChange is not a function."

Building the dropdown from a list

```
<select value={selectedCurrency} onChange={(e) => onCurrencyChange && onCurrencyChange(e.target.value)}>
  {currencyOptions.map((currency) => (
    <option key={currency} value={currency}>{currency}</option>
  ))}
</select>
```

`.map()` walks through every code in `currencyOptions` (e.g. `["usd", "inr", "eur"]`) and turns each one into an `<option>` element. The `key={currency}` is required by React whenever you render a list — it's an invisible label that helps React efficiently figure out which items changed, were added, or removed, instead of re-drawing the entire list every time.

5. App.jsx — Where Everything Comes Together

`App.jsx` is the "brain" of the whole application. It owns every piece of data the app needs to remember, and every function that changes that data. `InputBox` never stores anything permanently — it just displays what `App` tells it to and reports events back. This overall pattern is called **"lifting state up"**: when two sibling components (the From box and the To box) need to share or coordinate data, that data is stored one level higher, in their common parent.

The four pieces of state

```
const [amount, setAmount] = useState(0)
const [from, setFrom] = useState("usd")
const [to, setTo] = useState("inr")
const [convertedAmount, setConvertedAmount] = useState(0)
```

- **amount** — the number the user types into the "From" box. Starts at 0.
- **from** — which currency they're converting FROM. Starts at "usd".
- **to** — which currency they're converting TO. Starts at "inr".
- **convertedAmount** — the calculated result, shown (disabled) in the "To" box. Starts at 0.

Each of these is completely independent — that's why there are four separate `useState` calls instead of one big object. This keeps each update simple and avoids accidentally overwriting unrelated data.

Pulling in live exchange rates

```
const currencyInfo = useCurrencyInfo(from)
const currencyOptions = Object.keys(currencyInfo)
```

- `useCurrencyInfo(from)` — calls the custom hook you read about in Section 3, passing in whatever the current "from" currency is. It returns an object like `{ inr: 83.2, eur: 0.91, gbp: 0.79, ... }` — the value of 1 unit of from expressed in every other currency.
- Because this hook internally uses `useEffect` with `[currency]` as its dependency, changing from here (e.g. by picking a new "From" currency) automatically triggers a fresh fetch for the new base currency — App doesn't need to manually re-fetch anything itself.
- `Object.keys(currencyInfo)` — takes an object and returns just its key names as an array, e.g. `["inr", "eur", "gbp", ...]`. This becomes the list of choices shown in both dropdowns.

The `convert()` function

```
const convert = () => {
  setConvertedAmount(amount * currencyInfo[to])
}
```

Straightforward multiplication: take whatever amount the user typed, multiply it by the exchange rate for the currently selected "to" currency (looked up from the `currencyInfo` object), and store the result. This only runs when the form is submitted (see the form's `onSubmit` below) — not on every keystroke — so the displayed result only updates when the user actually clicks "Convert."

The `swap()` function — the trickiest part

```
const swap = () => {
  setFrom(to)
  setTo(from)
  setAmount(convertedAmount)
  setConvertedAmount(amount)
}
```

This flips everything: the "from" currency becomes the "to" currency and vice versa, and the amount swaps places with the converted amount (so if you had 100 USD → 8300 INR, swapping gives you 8300 INR → 100 USD as your new starting point).

Why doesn't this break, even though `setFrom()` is called before `setTo()` reads "from"?

This is a common point of confusion. In React, calling a state setter like `setFrom(to)` does **not** immediately change the `from` variable. All four lines in this function read the values of `from`, `to`, `amount`, and `convertedAmount` exactly as they were at the moment `swap` was called — before any of these updates take effect. React collects all four `set . . . ()` calls together and applies them in one batch, then re-renders once with all the new values. So the swap works correctly.

The JSX / layout, piece by piece

The rest of `App.jsx` is what gets displayed. It's built with Tailwind CSS, a utility library where each short class name maps to one CSS rule (instead of writing custom CSS files).

Tailwind class	What it does
<code>w-full h-screen</code>	Full page width and full viewport height
<code>flex flex-wrap justify-center items-center</code>	Centers the card both horizontally and vertically
<code>bg-cover bg-no-repeat</code>	Makes the background photo fill the screen without tiling
<code>backdrop-blur-sm bg-white/30</code>	Frosted-glass effect: blurs what's behind the card, 30% white tint
<code>rounded-lg</code>	Rounded corners
<code>absolute left-1/2 -translate-x-1/2</code>	Centers the swap button horizontally, independent of the flow

The form and the two `InputBox` usages

```
<form onSubmit={(e) => { e.preventDefault(); convert() }}>
  <InputBox
    label="From"
    amount={amount}
    onAmountChange={setAmount}
    onCurrencyChange={setFrom}
    currencyOptions={currencyOptions}
    selectedCurrency={from}
  />
  <button type="button" onClick={swap}>swap</button>
  <InputBox
    label="To"
    amount={convertedAmount}
    currencyOptions={currencyOptions}
    onCurrencyChange={setTo}
    selectedCurrency={to}
    amountDisable
  />
  <button type="submit">Convert {from.toUpperCase()} to {to.toUpperCase()}</button>
</form>
```

- **`e.preventDefault()`** — by default, submitting an HTML form reloads the whole page. This line stops that browser default so React can handle the submit itself.
- **`onAmountChange={setAmount}`** — notice this passes the state setter *function itself* as the prop, not a call to it. `InputBox` will call it later, whenever the user types, with the new number as the argument — that's exactly what `setAmount(newVa lue)` needs.

- **Top InputBox ("From")** — fully interactive: user can type an amount and change the currency. Both `onAmountChange` and `onCurrencyChange` are wired up.
 - **Bottom InputBox ("To")** — `amountDisable` is set (shorthand for `amountDisable={{true}}`), and no `onAmountChange` is passed at all, because this field should only ever show a computed result, never accept typing.
 - **`type="button"` on the swap button** — very important. Without `type="button"`, a button inside a form defaults to `type="submit"` and would trigger the form's submit (running `convert()`) every time you just wanted to swap.
 - **`type="submit"` on the Convert button** — this is the one button that's allowed to trigger the form's `onSubmit`, which calls `convert()`.
 - **`{from.toUpperCase()}`** — pure JavaScript inside JSX curly braces; any JS expression can go there. This just makes the button read "Convert USD to INR" instead of "Convert usd to inr".
-

6. Walking Through a Real User Action

Scenario A — Typing an amount

- User types "5" into the From box.
- InputBox's `onChange` fires → calls `onAmountChange(5)`, which is really `setAmount(5)` in App.
- App re-renders. The From InputBox now receives `amount={5}` as a prop, so the box visibly shows 5.

Scenario B — Changing the "From" currency

- User picks "eur" in the From dropdown.
- InputBox calls `onCurrencyChange("eur")` → really `setFrom("eur")`.
- App re-renders with `from = "eur"`. Since App calls `useCurrencyInfo(from)` on every render, the hook notices currency changed, and its `useEffect` re-fires to fetch fresh EUR-based rates.
- Once the fetch resolves, the hook calls its own `setData(...)`, causing another re-render — now `currencyInfo` holds EUR rates.

Scenario C — Clicking "Convert"

- Form's `onSubmit` fires → `e.preventDefault()` stops a page reload → `convert()` runs.
- `convert()` computes `amount * currencyInfo[to]` and calls `setConvertedAmount(...)`.
- App re-renders; the "To" InputBox receives the new `convertedAmount` as its (disabled) amount and displays it.

Scenario D — Clicking "swap"

- Button's `onClick` fires `swap()` (the form is not submitted, because this button has `type="button"`).
 - All four state values flip in one batch (see Section 5's note on why this is safe), and App re-renders with From/To currencies exchanged and the two amounts exchanged.
-

7. Quick-Reference Glossary

Term	Meaning
Component	A reusable, self-contained piece of UI written as a JavaScript function that returns JSX. App and InputBox are both components.
JSX	HTML-like syntax you write inside JavaScript. Gets compiled into plain JS function calls behind the scenes.
Props	Read-only inputs passed from a parent component into a child, e.g. label="From". A child can never change its own props.
State	Data a component owns and can change over time, created with useState. Changing it triggers a re-render.
Hook	A function starting with 'use' that taps into React features. useState, useEffect, and useId are all built-in hooks.
Custom hook	Your own function, named useSomething, that combines built-in hooks into reusable logic (e.g. useCurrencyInfo).
Controlled component	An input whose displayed value is fully driven by React state (via the value prop), not by the browser's own memory.
Lifting state up	Storing shared data in the nearest common parent (App) instead of duplicating it inside sibling children.
Re-render	React re-running a component's function to figure out what the UI should look like now, after state or props changed.
Dependency array	The [] second argument to useEffect; controls when the effect re-runs.

8. A Few Things Worth Noticing (Bonus)

- **No loading or error state.** While the exchange-rate fetch is in progress (or if it fails), currencyInfo is just an empty object — the dropdowns will briefly be empty and currencyInfo[to] could be undefined. A more robust version would track a loading flag and show a spinner or error message.
- **amount is a number, not always a valid one.** Typing nothing into a number input can produce NaN after Number("")-style conversion in edge cases — worth guarding against in a production version.
- **The whole app is 'uncontrolled' in one sense** — it always starts at USD → INR with amount 0 on every page refresh, since these are just useState initial values, not saved anywhere (like localStorage).
- None of this means the code is wrong — it works exactly as intended for a learning project. These are simply the next things you'd reach for once you're comfortable with what's already here.

That's the whole app — four files, each with one job, talking to each other through props, state, and one custom hook. If any single piece above is still fuzzy, the best next step is to open that exact file, find the matching section here, and read them side by side.